



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-11332 Ansible-core argument injection in ansible-galaxy role dependency installs

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-05
Version	1.1
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-11332 Ansible-core argument injection in ansible-galaxy role dependency installs - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-05 | TLP:CLEAR | Version: 1.1 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
8. Mitigation and Workarounds
9. Detection Engineering
10. Threat Hunting
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. References

CVE-2026-11332 Ansible-core argument injection in ansible-galaxy role dependency installs

1. Executive Summary

CVE-2026-11332 is a high-severity argument-injection flaw in `ansible-core` affecting how `ansible-galaxy role install` handles dependency specifications from a role's `meta/requirements.yml` file. A malicious role author can place git configuration arguments in the dependency `src` field so that a victim's workstation, CI runner, or automation host executes attacker-controlled commands during role installation.

The issue is not a remote network service exposure in the usual sense. Instead, it is a supply-chain and operator-workstation risk: exploitation requires a defender, administrator, or automation pipeline to install a malicious role or a role with a malicious transitive dependency. Once triggered, the injected git configuration can yield arbitrary code execution with the privileges of the user or service account running `ansible-galaxy`.

Confidence is high on the core finding because the Red Hat CNA record, the NVD entry, and upstream `ansible/ansible` fix and backport pull requests all align on the vulnerable behavior and remediation direction. The original intake source was an MSRC Update Guide relay entry, but client-facing conclusions in this report rely on the upstream CVE and maintainer artifacts rather than the Microsoft mirror.

2. Vulnerability Metadata

Field	Value
CVE	`CVE-2026-11332`
Product / component	`ansible-core`, specifically `ansible-galaxy role install` dependency processing
Vulnerable input path	role dependency definitions in `meta/requirements.yml`
Exploitable field	dependency `src` value passed to `git clone`
Weakness	`CWE-88` Improper Neutralization of Argument Delimiters in a Command (`Argument Injection`)
Severity	High / Red Hat `Important`
CVSS v3.1 vector	`AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H` (`7.8 / HIGH`)
CNA / public disclosure date	`2026-06-05`
NVD publication date	`2026-06-05`
Bug tracker	`RHBZ#2485379`
Upstream fix PR	`ansible/ansible#87070`
Upstream follow-up test PR	`ansible/ansible#87085`
Backport activity observed	stable branch pull requests opened for `stable-2.16` through `stable-2.21`

Exact vulnerable and fixed packaged version ranges were not stated in the CNA or NVD records reviewed here. Defenders should therefore track vendor package advisories and upstream release notes closely rather than assuming a specific branch is already remediated.

3. Source Review & Confidence

Source	Contribution	Confidence
Reviewed source	Syndicated relay entry for `CVE-2026-11332` with the headline `Ansible-core: argument injection in ansible-galaxy role i	Useful intake and cross-platform visibility source, but not authoritative for Ansible product impact
Reviewed source	Red Hat CNA record states that a malicious role author can inject arbitrary git configuration flags through a dependency	Strongest source for the authoritative vulnerability description and timeline.
Reviewed source	NVD mirrors the vulnerability description, lists CVSS v3.1 vector `AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H` (`7.8 / HIGH`),	Strong corroborating source for structured metadata; record was `Awaiting Analysis` at review time.
Reviewed source	Red Hat rates the issue `Important`, repeats the arbitrary-code-execution description, credits Tommaso Bona, and marks `	Primary downstream vendor metadata source for severity and package-state context.
Reviewed source	Public Red Hat Bugzilla record tracks the issue as `ansible-core: argument injection in ansible-galaxy role install lead	Useful corroborating tracking artifact tied directly to the CNA record.
Reviewed source	Upstream fix PR states that role requirement `src` and `name` are now passed as positional arguments (`git clone -- <src	Strongest technical remediation source because it exposes the exact code-level fix and the new secur
Reviewed source	Follow-up test PR fixes integration test behavior after `#87070`; related stable-branch backport PRs `#87074` through `#	Good maintainer signal for branch coverage and regression testing, but not the primary source for in

Confidence is high on the vulnerability mechanics and impact: Confidence is lower on exact downstream version exposure because public vendor package matrices were limited at review time. The MSRC Update Guide item proved the CVE was being syndicated broadly, but it is not the authoritative source for product impact because the vulnerable software is Ansible rather than a Microsoft product.

4. Vulnerability Details

The vulnerable path is in `ansible-galaxy role install` when a role declares dependencies in `meta/requirements.yml`. If a dependency uses a git-backed source, `ansible-galaxy` eventually constructs a `git clone` command using values derived from that dependency metadata.

Before the upstream fix, the code appended the dependency `src` and destination `name` directly to the git command. That allowed a malicious dependency source beginning with crafted argument content to be interpreted as git options rather than as a repository location.

The upstream fix changes the command construction from effectively:

- `git clone <src> <name>`

to:

- `git clone -- <src> <name>`

That separator forces subsequent values to be treated as positional arguments. The added integration test demonstrates the core abuse case by attempting to inject `-c core.sshCommand=...` through a malicious dependency value and then verifying the injected command does not execute after the fix.

Operationally, this means the security boundary is broken at install time. A victim does not need to run a playbook from the malicious role; installing the role and allowing Ansible to resolve its dependencies can be enough to trigger the attacker-controlled behavior.

5. Attack Scenarios

1. An administrator installs a third-party role from a public repository or internal share, and that role contains a malicious dependency entry in `meta/requirements.yml`.
2. A CI pipeline automatically resolves role dependencies during a build or test stage, giving an attacker code execution on the runner if the pipeline consumes an untrusted or compromised role.
3. A threat actor compromises an otherwise trusted role source and adds a malicious transitive dependency so defenders execute the payload during what appears to be a routine `ansible-galaxy role install` workflow.
4. A developer or operator reviews only the top-level role content and misses the nested dependency `src` entry, assuming installation is safe because the visible role logic appears benign.

6. Threat Landscape & Exploitation Status

- No authoritative source reviewed here states that `CVE-2026-11332` is currently exploited in the wild.
- The CISA ADP enrichment in the CVE record marks `Exploitation: none`, `Automatable: no`, and `Technical Impact: total`, which supports prioritizing the flaw for high-impact local compromise without overstating current public exploitation.
- The exploit path is realistic for environments that routinely install external roles or resolve dependencies in unattended automation pipelines.
- Because the bug sits in a software supply-chain workflow, organizations that treat infrastructure-as-code content as trusted by default should consider the exposure more seriously than the `AV:L` vector alone might imply.
- Upstream's rapid publication of a dedicated security fix and multiple stable-branch backports indicates maintainers view remediation as important across active release lines.

7. Affected Products and Exposure

Directly affected products and workflows include:

- Systems running vulnerable `ansible-core` builds that perform `ansible-galaxy role install` against roles with dependency metadata.
- Red Hat Ansible Automation Platform 2 environments where the packaged `ansible-core` remains unfixed.
- CI/CD runners, build agents, and admin workstations used to validate or stage Ansible roles.
- Internal artifact pipelines that mirror or ingest third-party roles without screening dependency metadata.

Higher-risk patterns:

- Installing roles from public Galaxy or unvetted third-party repositories.
- Allowing automated dependency resolution from external git sources.
- Running `ansible-galaxy` on privileged jump hosts, workstation accounts with broad SSH access, or CI runners holding cloud credentials.
- Treating transitive dependencies as implicitly trusted because only the top-level role source is reviewed.

Lower-risk patterns:

- Environments that do not use `ansible-galaxy role install` for third-party roles.
- Workflows that pin roles to vetted internal mirrors and review dependency metadata before promotion.
- Hosts updated to a vendor or upstream release that includes the `git clone -- <src> <name>` remediation.
- Sandboxed installation pipelines with minimal privileges and no sensitive credential material.

8. Mitigation and Workarounds

Priority actions:

5. Apply vendor package updates or upstream releases that incorporate the `CVE-2026-11332` fix as soon as they are available for your supported Ansible branch.
6. Until patching is confirmed, avoid installing roles from untrusted or newly introduced sources, especially when they can carry transitive dependency definitions.
7. Review `meta/requirements.yml` for any role before installation and treat dependency `src` values beginning with `-`, containing git `-c` options, or otherwise resembling command-line flags as suspicious.
8. Run `ansible-galaxy` installs in low-privilege, disposable build environments rather than administrator workstations or long-lived automation hosts.
9. Restrict outbound network and credential access from CI runners or build containers that must resolve role dependencies.
10. Track Red Hat, downstream Linux distribution, and upstream Ansible release advisories so you can map the fix to actual packaged versions in your estate.

Red Hat's CNA record states that no formal mitigation meeting Red Hat Product Security criteria was available at publication time. The defensive guidance above should therefore be treated as risk-reduction measures rather than as a vendor-declared workaround.

9. Detection Engineering

Detection should focus on role-install activity, suspicious git argument patterns, and child-process behavior from Ansible tooling:

Signal	Telemetry Source	Analytic Goal
Alert on `ansible-galaxy` role install` executions on admin workstations, CI runners, or automation servers that do not normally install new roles.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity
Monitor process command lines for `git clone` launched by `ansible-galaxy` where git configuration flags such as `-c` appear near repository-source arguments.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity
Hunt for unexpected child processes spawned from `git`, `ssh`, or shell interpreters immediately after role dependency installation attempts.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity
Log and review changes to internal role mirrors, especially updates to `meta/requirements.yml` and dependency `src` fields.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity
Add static screening for dependency entries whose `src` value begins with a dash or contains suspicious git configuration strings such as `core.sshCommand`.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity

9.1 Detection Query

```
// Build and package provenance pivots for CVE-2026-11332 Ansible-core argument injection in ansible-galaxy role dependency installs
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ProcessCommandLine has_any ("go version", "go env", "pip show", "pip list", "syft", "grype", "sbom")
| project Timestamp, DeviceName, InitiatingProcessAccountName, FileName, ProcessCommandLine
```

```
| order by Timestamp desc
```

9.2 Exposure / Triage Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine
has_any ("panic", "fatal error")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

10. Threat Hunting

Suggested hunt questions:

- Which hosts in the environment have executed `ansible-galaxy role install` within the last 30 days?
- Which internal repositories or mirrored roles contain `meta/requirements.yml` files with dependency `src` values pointing to external git locations?
- Do any dependency `src` values begin with `~`, contain `~c`, or reference unusual git configuration keys such as `core.sshCommand`?
- Did any CI runners or admin workstations spawn unexpected `git`, `ssh`, or shell subprocesses during Ansible role installation jobs?
- Which Ansible automation hosts still carry package versions published before the upstream fix and have not yet been mapped to a vendor advisory?
- Have any recently onboarded or updated third-party roles introduced new transitive dependencies that were not part of the previous reviewed baseline?

10.1 Hunt Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine
has_any ("panic", "fatal error")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

11. MITRE ATT&CK Mapping

The strongest evidence-supported ATT&CK mappings for this vulnerability's exploitation path are:

- `T1195.001` Compromise Software Dependencies and Development Tools: a malicious role author or compromised role source can weaponize Ansible dependency metadata so the victim executes attacker-controlled behavior during dependency resolution.
- `T1204` User Execution: exploitation requires a user or automated workflow to install the malicious role or dependency chain.
- `T1059` Command and Scripting Interpreter: the demonstrated abuse path uses injected git configuration to trigger attacker-controlled command execution on the victim host.

Additional post-exploitation techniques would depend on what the attacker does after initial execution and are therefore outside the strongest public evidence base reviewed for this report.

12. Validation / Lab Testing

This review validated the draft against public advisory, CVE, and upstream maintainer artifacts rather than reproducing the exploit locally. Analyst validation steps completed:

- Reviewed the MSRC Update Guide intake item to confirm the CVE and relay headline.

- Queried the CVE.org / Red Hat CNA record for the authoritative description, timeline, CWE, and CISA ADP enrichment.
- Queried the NVD JSON 2.0 API for corroborating description, CVSS vector, and reference metadata.
- Queried Red Hat's security data API and public Bugzilla record for severity, package state, acknowledgement, and public tracking metadata.
- Reviewed upstream pull request `ansible/ansible#87070` to validate the exact remediation pattern (`git clone -- <src> <name>`).
- Reviewed upstream pull request `ansible/ansible#87085` and stable branch backport pull requests to confirm maintainer test coverage and branch remediation activity.

No Lost Boys Cyber proof-of-concept execution was performed during this review, and no claim is made here that exploitation was independently reproduced in a local lab.

13. Recommended Actions Summary

Priority	Owner	Action
High	Security / Platform Owners	Prioritize `CVE-2026-11332` for admin workstation and CI pipeline review even though the CVSS vector is local, because the practical risk is supply-chain-triggered code execution during routine automation workflows.
Critical	Platform / Application Owners	Patch or upgrade every supported Ansible environment once a vendor or upstream release carrying the fix is available for that branch.
High	Security / Platform Owners	Freeze or tightly review new third-party role onboarding until dependency metadata screening is in place.
High	Security / Platform Owners	Move role-install workflows into disposable, low-privilege environments and minimize secrets exposure on systems that must resolve dependencies.
High	SOC / Detection Engineering	Hunt for suspicious `meta/requirements.yml` dependency sources and anomalous `git` child-process behavior tied to recent `ansible-galaxy` activity.

14. References

11. Microsoft Security Response Center Update Guide relay entry, `CVE-2026-11332`
12. CVE.org / Red Hat CNA record, `CVE-2026-11332`
13. NVD, `CVE-2026-11332`
14. Red Hat security data API, `CVE-2026-11332`
15. Red Hat Bugzilla, `RHBZ#2485379`
16. ansible/ansible pull request `#87070`, `Fix CVE-2026-11332 malformed role requirements`
17. ansible/ansible pull request `#87085`, `Fix ansible-galaxy-role test`
18. ansible/ansible stable-branch backport pull requests `#87074` through `#87078`